
<SJTU>

< **GitHub** 仓库问答与 **Issue** 分析系统>
软件架构文档

版本 <1.1>

<项目名称>	Version: <1.1>
软件架构文档	Date: <21/07/2026>

修订历史记录

日期	版本	说明	作者
<17/07/2026>	<1.0>	<完成架构文档 V1.0，完成用例、逻辑、技术三类核心视图的 UML 建模与文字说明；补充部署、进程、数据视图设计；其余可选视图标注省略理由>	<李明睿 林圣>
<21/07/2026>	<1.1>	<在 V1.0 的基础上，修正了一些已知问题，完善了业务服务层所用的 api 描述，完善了可维护性视图，对所用算法进行了较详细的描述>	<李明睿 林圣>

<项目名称>	Version: <1.1>
软件架构文档	Date: <21/07/2026>

目录

1. 简介	错误!未定义书签。
2. 用例视图	4
3. 逻辑视图	6
4. 部署视图（可选）	11
5. 进程视图（可选）	12
6. 实现视图（可选）	12
7. 技术视图	12
8. 数据视图（可选）	13
9. 算法视图（可选）	16
10. 性能视图（可选）	17
11. 可靠性视图（可选）	17
12. 安全性视图（可选）	17
13. 易用性视图（可选）	17
14. 可维护性视图（可选）	17

<项目名称>	Version: <1.1>
软件架构文档	Date: <21/07/2026>

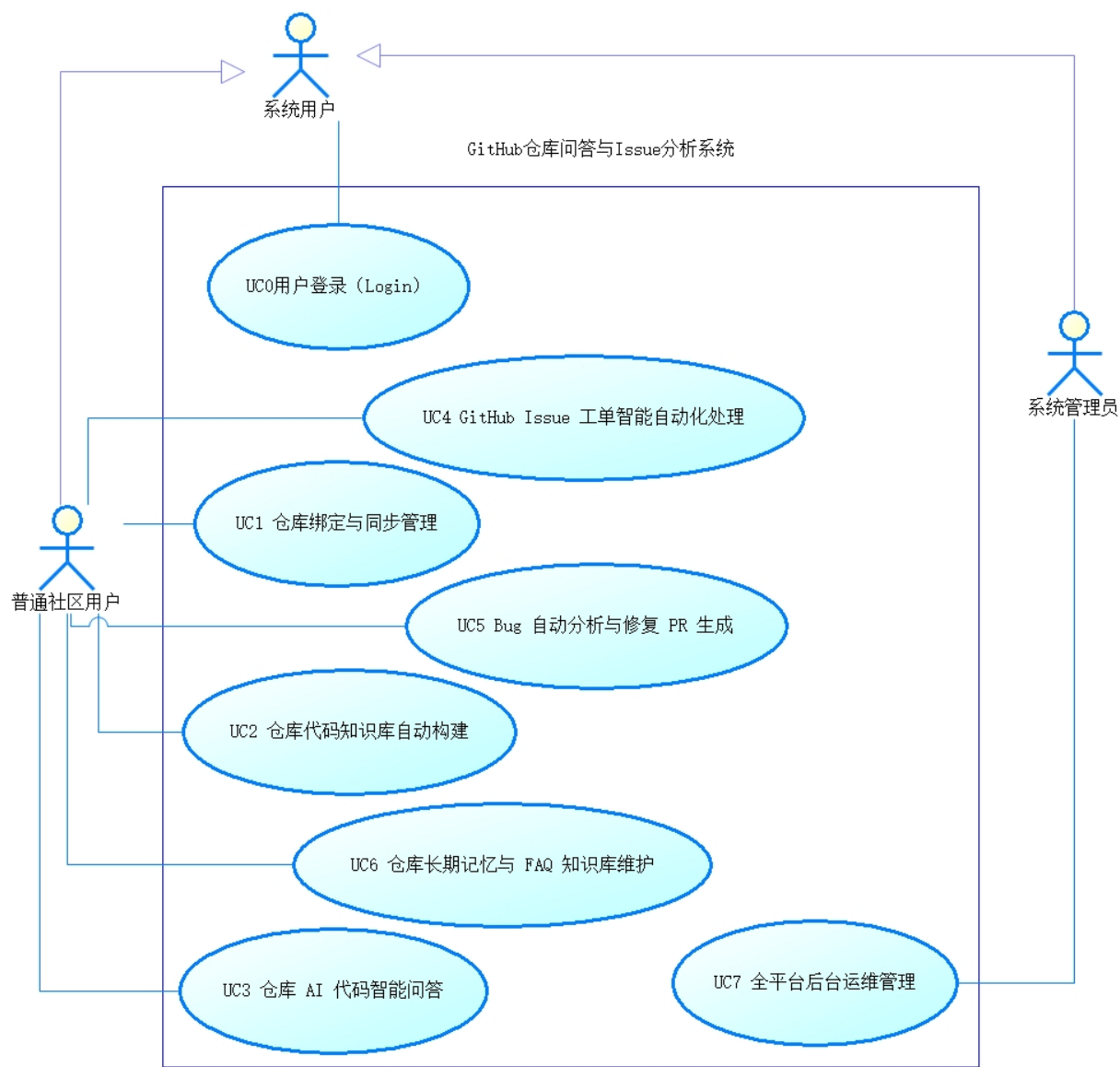
软件架构文档

1. 简介

1.1 目的

本文档将从构架方面对系统进行综合概述，其中会使用多种不同的构架视图来描述系统的各个方面。它用于记录并表述已对系统的构架方面作出的重要决策。

2. 用例视图



软件架构的用例视图描述。用例视图是选择迭代关注的模型场景和/或用例的重要输入。它描述了一组代表系统核心、关键功能的场景和/或用例。此外，它还描述了具有实质性架构覆盖范围用例（这些用例能够锻炼许多架构元素，或者能够强调、说明架构中的特定敏感、微妙点）。

<项目名称>	Version: <1.1>
软件架构文档	Date: <21/07/2026>

本系统的用例包括：

- UC0 用户登录 (Login)
- UC1 仓库绑定与同步管理
- UC2 仓库代码知识库自动构建
- UC3 仓库 AI 代码智能问答
- UC4 GitHub Issue 工单智能自动化处理
- UC5 Bug 自动分析与修复 PR 生成
- UC6 仓库长期记忆与 FAQ 知识库维护
- UC7 全平台后台运维管理

这些用例由普通社区用户或系统管理员发起。此外，在系统运行期间，还会涉及到与外部系统（外部 GitHub 平台 API、Git 工具以及大语言模型 API）的交互。

2.1 具有架构意义的用例

2.1.1 UC0 用户登录 (Login)

简要描述：此用例描述普通社区用户或系统管理员如何登录到 GitHub 仓库问答与 Issue 分析系统。该用例支持通过 GitHub OAuth 授权或常规账号密码进行安全鉴权，并为后续的个性化仓库管理和多轮会话存储提供必要的上下文凭证。启动该用例的参与者为系统用户（包括普通社区用户和系统管理员）。

2.1.2 UC1 仓库绑定与同步管理

简要描述：此用例允许用户绑定其拥有访问权限的公开或私有 GitHub 仓库，并配置数据同步策略。系统将根据配置周期性地拉取最新的代码、README、Issues、Pull Requests 等元数据。该用例的启动者是普通社区用户。外部的 GitHub API 是该用例涉及的核心外部系统参与者。

2.1.3 UC2 仓库代码知识库自动构建

简要描述：在仓库绑定或检测到代码同步更新后触发。此用例将拉取的源码和文档进行清洗、解析、切片，通过 Embedding 模型将文本块转化为向量，并最终持久化至向量数据库中。由于该用例涉及大量文件的 I/O 操作与高并发向量计算，在架构上通过后台异步任务流水线执行。普通社区用户通过触发同步间接启动该用例。

2.1.4 UC3 仓库 AI 代码智能问答

简要描述：此用例允许用户使用自然语言对已绑定的仓库进行智能提问。系统通过混合检索（向量相似度检索与文本关键词过滤）获取相关代码片段和文档作为上下文，结合大语言模型生成具有可追溯引用来源的回答，并支持多轮对话。该用例的启动者是普通社区用户。外部大语言模型（LLM）API 和向量数据库是协同运行的外部系统参与者。

2.1.5 UC4 GitHub Issue 工单智能自动化处理

简要描述：此用例支持对仓库中的 Issue 进行智能化分析。系统获取指定 Issue 的文本、讨论历史后，自动生成 Issue 摘要、判断工单分类、检测是否存在重复 Issue，并分析其可能的原因（Root Cause）。该用例由普通社区用户主动触发，或由配置的 GitHub Webhook 在产生新 Issue 时自动回调触发。

2.1.6 UC5 Bug 自动分析与修复 PR 生成

简要描述：此用例允许系统在检测到特定 Bug 类型或代码缺陷后，自动分析相关文件和提交记录，生成推荐的修复方案（Recommended Fix），并自动在 GitHub 远程仓库中为该分支创建对应的 Pull Request（PR）以供人类开发者审核。该用例由普通社区用户触发，并高度依赖外部 GitHub 平台的写操作权限。

2.1.7 UC6 仓库长期记忆与 FAQ 知识库维护

简要描述：此用例支持用户对仓库的 FAQ 和长期记忆进行维护。通过将高质量的问答历史或代表性的经典 Issue 讨论沉淀为静态 FAQ，提高相似问题的检索效率，并降低未来调用大语言模型

<项目名称>	Version: <1.1>
软件架构文档	Date: <21/07/2026>

API 的开销。该用例由普通社区用户启动。

2.1.8 UC7 全平台后台运维管理

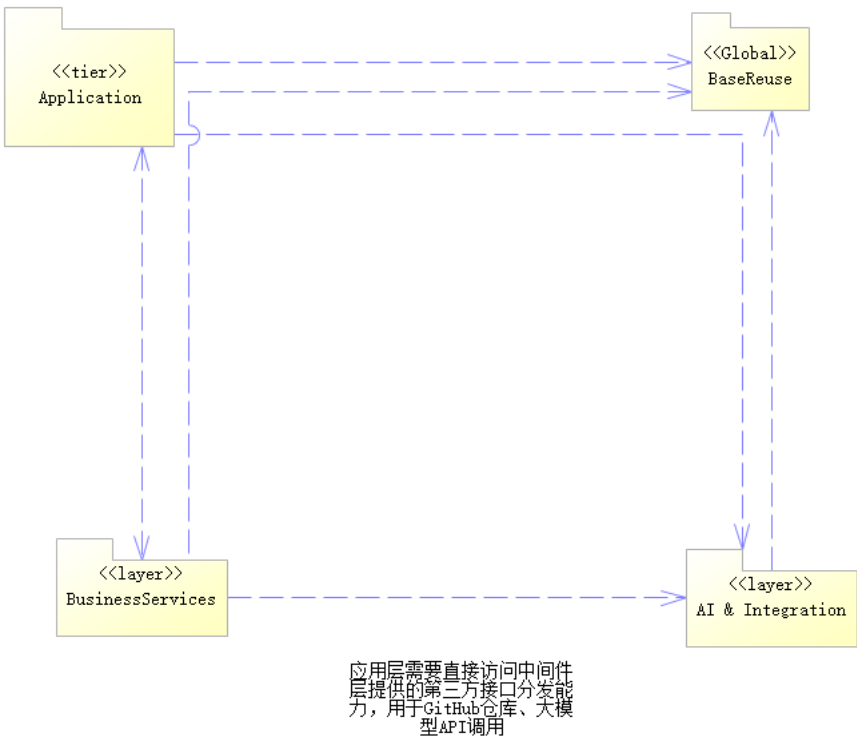
简要描述：此用例允许系统管理员对平台进行全局的配置与运维管理。主要包括系统参数配置、用户及仓库配额限制、外接大模型 API 的调用额度与密钥监控、系统运行日志分析以及对账审计。该用例的唯一启动者是系统管理员。

3. 逻辑视图

对 GitHub 仓库 AI 问答系统架构逻辑视图进行描述，介绍系统核心包、子系统分层组织方式，同时说明各层支撑的核心用例实现逻辑。本章配套包图，展示架构关键包、子系统、分层之间的依赖关系。

本系统逻辑架构划分为 4 个核心包：Application、Business Services、MiddlewareAI、Base Reuse，整体遵循分层单向依赖架构；所有用户交互、仓库同步、AI 问答、后台运维的业务用例，均由四层包内的边界类、控制类、实体类协作完成。

3.1 架构总览 —— 包与子系统分层



系统整体四层分层架构总览：应用层是所有用户交互的入口；业务服务层承载系统全部核心业务逻辑；AI 层统一封装 GitHub 平台、大模型、向量数据库等第三方外部服务；基础复用底层提供全局通用工具与数据持久化支撑。

整体依赖规则：上层分层单向依赖下层分层，基础复用底层不依赖任何上层模块，是整个架构的最底层支撑。

3.1.1 应用层

应用层存放所有面向用户交互的界面边界类，内部拆分为两个独立前端子包：普通用户前台页面包、管理员后台页面包，覆盖账号登录、仓库绑定同步、AI 代码问答、Issue 工单处理、后台数据统计等全部页面

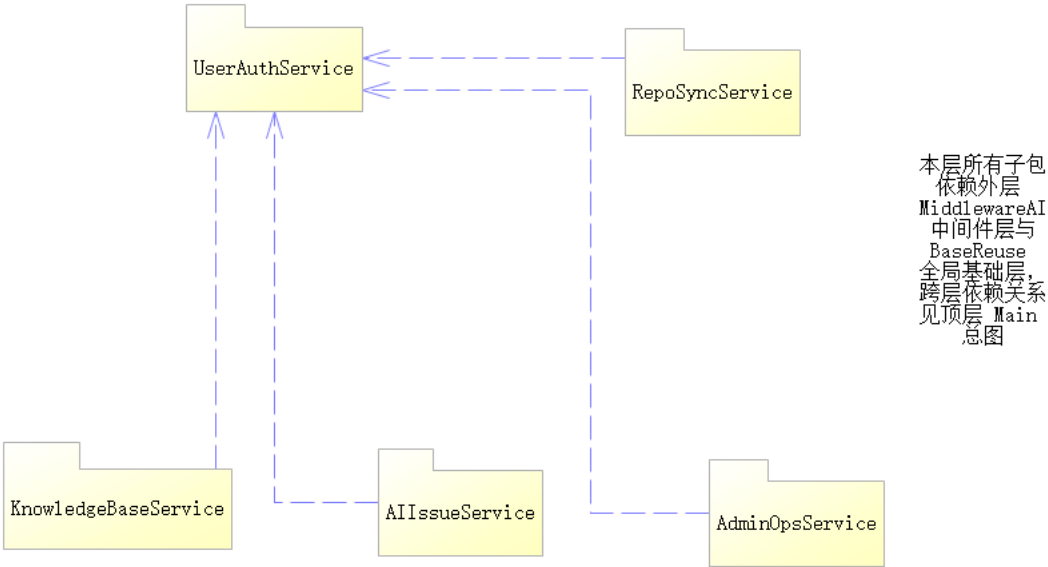
<项目名称>	Version: <1.1>
软件架构文档	Date: <21/07/2026>

操作场景。
本层仅负责接收、转发用户操作请求，不处理任何业务逻辑，单向依赖业务服务层，同时全局复用基础复用底层提供的通用工具。



本层两个前端子包无内部互相调用依赖；所有页面发起的业务请求均向下依赖顶层 Main 总图中的 BusinessServices 业务服务层；同时可直接跨层调用 MiddlewareAI 中间件层提供的 GitHub、大模型第三方接口；全层所有组件全局复用 BaseReuse 基础复用层通用工具类

3.1.2 业务服务层
业务服务层是整个系统的业务核心，存放所有业务用例对应的控制类，覆盖用户鉴权、仓库代码同步、知识库构建、AI 问答交互、管理员后台运维全流程功能。本层依赖 AI 中间件层与基础复用底层，不会直接调用第三方接口、操作底层数据库。



用户与

1. 认证服务 (UserAuthService)
用户与认证服务主要负责前端的鉴权和基本用户设置的管理。该服务接收来自前端的请求来处理 GitHub OAuth 的登录授权，具体通过路径 /auth/github 实现，这也是系统交互的起点。登录成功并完成会话建立

<项目名称>	Version: <1.1>
软件架构文档	Date: <21/07/2026>

后，服务会暴露接口来支持获取当前认证用户的个人 Profile 数据，例如头像与用户名，其对应路径为 /api/user/profile。此外，用户的全局大模型配置同样由该服务接管，开发者和用户可以使用 /api/user/setting/llmconfig 路径来拉取或通过 /api/user/setting/llmconfig/set 来保存其对应大模型的 API Key、模型名称及 Base URL 信息。由于其他业务服务在调用 AI 引擎或拉取数据时，往往需要校验用户的 Token 或读取其配置的 API 密钥，因此该服务成为整体架构底座，包图中其他子包也全部指向该服务。

```
"paths": {
  "/auth/github": {
    "get": {
      "tags": ["Auth"],
      "summary": "Start GitHub OAuth login",
      "description": "Browser navigation (not apiFetch). Redirects to GitHub authorize, then backend completes via `auth/callback` and returns the user to the frontend OAuth success page with token query params.",
      "operationId": "startGitHubLogin",
      "responses": {
        "302": {
          "description": "Redirect to GitHub OAuth authorize URL"
        },
        "500": {
          "description": "OAuth env (client id/secret) not configured",
          "content": {
            "application/json": {
              "schema": { "$ref": "#/components/schemas/ErrorDetail" }
            }
          }
        }
      }
    }
  }
}
```

2. 仓库同步服务 (RepoSyncService)

仓库同步服务专注于处理用户相关的 GitHub 仓库发现与数据同步维护。这一服务的交互主要集中在两大核心功能点上：前端通过 /api/repos 路径拉取当前认证用户在平台绑定的所有代码仓库列表，此时获取的数据包里会携带仓库同步状态和上次同步的时间戳；如果需要聚焦于某一个具体的仓库，系统会利用路径 /api/repos/{repoId} 获取更精细的数据。在这两种场景中，系统会返回对应仓库的主开发语言、星标情况以及是正在同步、已经同步完毕，抑或是同步失败状态，保障上层数据展现的时效性。

```
"/api/repos": {
  "get": {
    "tags": ["Repositories"],
    "summary": "List repositories for the authenticated user",
    "operationId": "fetchRepoList",
    "security": [{ "bearerAuth": [] }],
    "responses": {
      "200": {
        "description": "Repository list",
        "content": {
          "application/json": {
            "schema": { "$ref": "#/components/schemas/RepositoryListResponse" }
          }
        }
      },
      "401": { "$ref": "#/components/responses/Unauthorized" },
      "500": { "$ref": "#/components/responses/ServerError" }
    }
  }
}
```

3. 知识库服务 (KnowledgeBaseService)

知识库服务是处理仓库知识资产索引构建的核心单元，它提供了从策略管理到实际索引生成的闭环机制。前端可以先通过 /api/repos/{repoId}/knowledge/policy 路径读取有关索引限额和排除规则的策略定义。在准备好构建条件后，用户可以使用 /api/repos/{repoId}/knowledge/build 接口提交一个包含最大提交数和索引要求的请求来触发后台构建。构建好的数据在 /api/repos/{repoId}/knowledge 路径进行统一展现，此时能够查看一个整合了知识树目录、模块划分、运行语言和共享 Deduplication 数据大盘的概要。不仅如此，系统还允许使用 /api/repos/{repoId}/knowledge/commits 获取已经编入索引的所有历史提交清单，并通过在 /api/repos/{repoId}/knowledge/compare 路径里传入两个哈希值对比任意两个提交节点的知识库变动。最后，用户还可以利用 /api/repos/{repoId}/knowledge/settings 随时修改此仓库的活动代码分支或配置索引参数。

<项目名称>	Version: <1.1>
软件架构文档	Date: <21/07/2026>

```
"/api/repos/{repoId}/knowledge/policy": {
  "get": {
    "tags": ["Knowledge"],
    "summary": "Get knowledge-indexing policy",
    "operationId": "fetchKnowledgePolicy",
    "security": [{ "bearerAuth": [] }],
    "parameters": [{ "$ref": "#/components/parameters/RepoId" }],
    "responses": {
      "200": {
        "description": "Indexing policy and limits",
        "content": {
          "application/json": {
            "schema": { "$ref": "#/components/schemas/KnowledgePolicy" }
          }
        }
      },
      "401": { "$ref": "#/components/responses/Unauthorized" }
    }
  },
}
```

4. AI 议题与智能问答服务 (AIIssueService)

AI 议题与智能问答服务将大模型的语义理解能力引入到了日常的 Issue 追踪与 Q&A 中。日常的管理层面上，服务通过 /api/repos/{repoId}/issues 和 /api/repos/{repoId}/issues/{issueNumber} 分别提供当前仓库中各类议题的列表调取和单条追踪，需要注意的是，在此设计中 PR 编号会被判定为无效而返回错误。当需要针对某一具体 Issue 寻找突破口时，前端会通过 /api/issues/analyze 将具体议题推送给后台进行智能诊断，后台将基于 heuristics 和大语言模型综合给出分析意见、修复信心、建议回复以及潜在涉及的代码变更范围。而针对更大范围的仓库整体答疑，则是借助 /api/chat 这个通用的 RAG 问答会话接口来实现，允许用户随时用自然语言进行交流，并返回带有准确行级文件引用的回答。

```
"/api/repos/{repoId}/issues": {
  "get": {
    "tags": ["Issues"],
    "summary": "List issues in a repository",
    "operationId": "fetchRepositoryIssues",
    "security": [{ "bearerAuth": [] }],
    "parameters": [
      { "$ref": "#/components/parameters/RepoId" },
      {
        "name": "state",
        "in": "query",
        "schema": {
          "type": "string",
          "enum": ["open", "closed", "all"],
          "default": "all"
        }
      }
    ],
    "responses": {
      "200": {
        "description": "Issue list (PRs excluded)",
        "content": {
          "application/json": {
            "schema": { "$ref": "#/components/schemas/IssueListResponse" }
          }
        }
      },
      "401": { "$ref": "#/components/responses/Unauthorized" },
      "404": { "$ref": "#/components/responses/NotFound" },
      "500": { "$ref": "#/components/responses/ServerError" }
    }
  },
}
```

5. 管理运维与操作执行服务 (AdminOpsService)

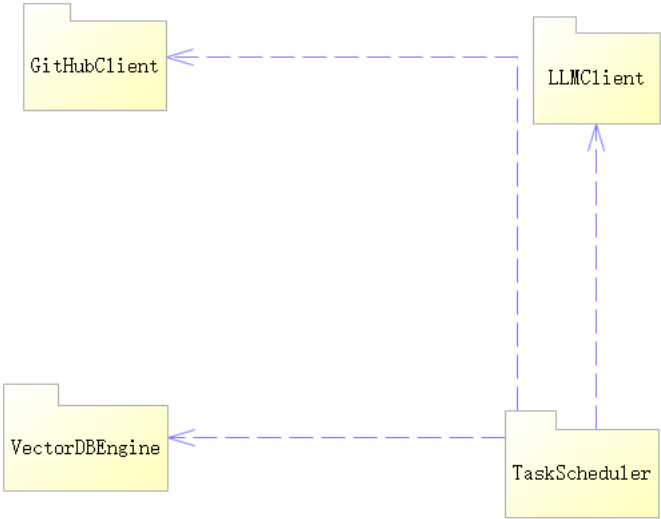
管理运维与操作执行服务代表着对系统和仓库的高阶管理与代码写入动作。服务在宏观上提供了 /api/portfolio/overview 路径，用于支持项目集层面的大盘多维汇总，给出多仓库的语言占比分布、大盘 Timeline 进度及总体运行指标，适合管理者鸟瞰全局。在具体的微观动作上，该服务通过 /api/repos/{repoId}/actions 直接接收诸如分支创建、代码文件提交或 PR 触发之类的结构化业务命令；同样，它也通过 /api/repos/{repoId}/actions/nl 接口支持更前沿的自然语言操作模式，让开发人员能够将自己的业务意图直接输入并交由大模型转换成底层的一连串 Git 指令执行。

<项目名称>	Version: <1.1>
软件架构文档	Date: <21/07/2026>

```
"/api/portfolio/overview": {
  "get": {
    "tags": ["Portfolio"],
    "summary": "Multi-repo portfolio overview",
    "operationId": "fetchPortfolioOverview",
    "security": [{ "bearerAuth": [] }],
    "responses": {
      "200": {
        "description": "Portfolio summary",
        "content": {
          "application/json": {
            "schema": { "$ref": "#/components/schemas/PortfolioOverview" }
          }
        }
      },
      "401": { "$ref": "#/components/responses/Unauthorized" },
      "500": { "$ref": "#/components/responses/ServerError" }
    }
  }
},
}
```

3.1.3 AI 层

AI 层封装全部外部第三方服务的调用逻辑，包含 GitHub 开放平台接口、通义千问大模型、Milvus 向量数据库、定时任务调度四大模块；该层隔离第三方服务的接口版本差异，向上层业务提供标准化调用能力，仅依赖基础复用底层。

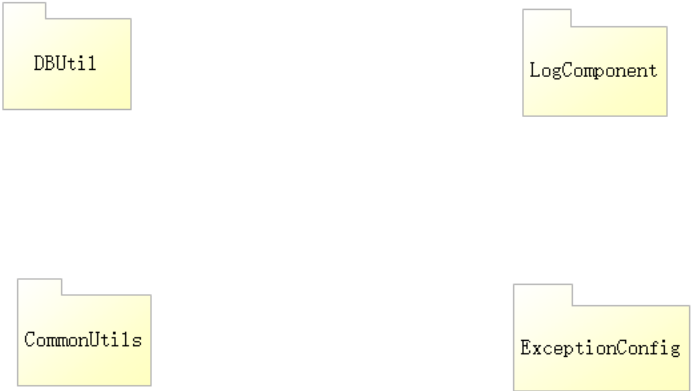


本层全部技术子包依赖外层BaseReuse全局基础层；本层仅
向上层 BusinessServices
业务服务层提供技术调用能力，完整跨层依赖关系查看顶层
Main 总图

3.1.4 基础复用底层

基础复用底层分为两大核心组件：一是数据访问组件，封装 MySQL 关系数据库、Milvus 向量数据库的数据实体与增删改查持久化操作；二是全局通用工具组件，包含日志打印、数据加密、全局异常处理、请求参数校验、系统配置等通用工具类。本层为所有上层分层提供底层支撑，不存在对外依赖。

<项目名称>	Version: <1.1>
软件架构文档	Date: <21/07/2026>



画布右侧 Text: 本层为全局公共底层组件，无下层依赖，被 Application、BusinessServices、MiddlewareAI 三层全部复用，跨层依赖见顶层 Main 总图

4. 部署视图（可选）

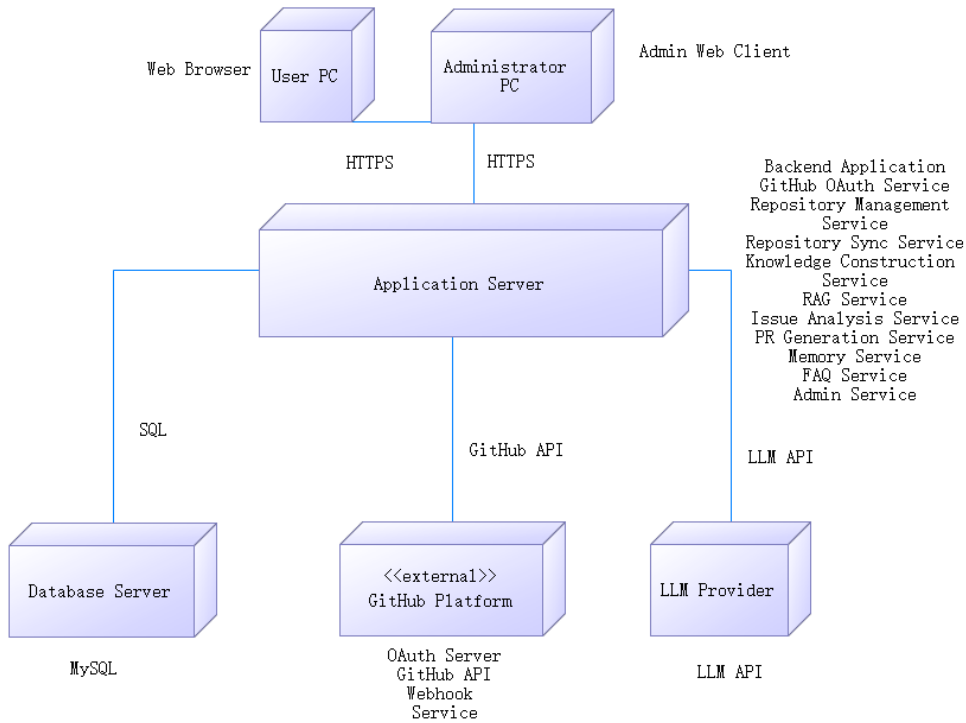


Diagram Name: Deployment View

<项目名称>	Version: <1.1>
软件架构文档	Date: <21/07/2026>

4.1 用户个人电脑 (User PC)

普通社区用户使用外部个人电脑，通过 HTTPS 协议连接至后端应用服务器。用户通过这些终端进行 GitHub 仓库绑定、发起 AI 问答、处理 Issue 工单以及查看自动修复的 PR 记录。

4.2 管理终端 (Admin PC)

系统管理员通过连接到内部局域网的本地管理终端直接访问后端应用服务器。管理员使用这些终端来监控全局同步日志、检验平台数据完整性、批量导出 FAQ 文档以及排查仓库同步故障。

4.3 后端应用服务器 (Application Server)

后端应用服务器是系统的主业务处理服务器。所有普通用户、管理员以及外部接口都通过互联网或局域网访问该服务器。它托管了系统的核心业务逻辑，包括 GitHub OAuth 认证、仓库同步、代码知识库构建、RAG 检索、Issue 智能分析以及自动修复 PR 生成等服务。

4.4 数据库服务器 (Database Server)

数据库服务器是保存系统所有结构化数据与代码向量知识的内部服务器。它包含用于存储用户信息、对话历史和工单记录的 MySQL 数据库，以及用于存储代码特征向量和长期记忆的 Milvus/ChromaDB 向量数据库。后端应用服务器通过内部网络 (LAN) 对其进行高频数据读写与向量检索。

4.5 GitHub 外部平台 (GitHub Platform)

GitHub 外部平台是提供代码托管与协作的外部系统。后端应用服务器通过 GitHub API 和 Webhook 接收服务与该平台进行交互，以实现用户身份授权、仓库代码及 Issue/PR 数据的同步，并在修复完成后自动推送分支和发起 Pull Request。

4.6 外部大模型服务 (LLM Provider)

外部大模型服务是由第三方（如 OpenAI 等）托管的外部 AI 推理平台。后端应用服务器通过安全的 HTTPS 接口调用其大模型 API，以实现代码语义解释、Issue 智能分类、回复生成以及代码修复方案生成。

5. 进程视图 (可选)

本系统为单体 Spring Boot 架构，后端业务运行在单一 JVM 进程，不存在多进程、分布式跨进程通信场景。系统并发能力由框架内置线程池提供，无自定义复杂进程协同设计。架构核心信息可通过其余视图完整表达，因此省略进程视图。

6. 实现视图 (可选)

本项目采用统一的软件工程项目结构进行开发，系统未划分为多个独立安装包或可独立部署的软件单元，而是作为一个完整应用进行构建和部署。因此，实现视图中不存在多个安装包之间的依赖关系，也无需对安装包组织结构进行单独描述。项目的源代码目录结构及模块划分已在前述逻辑视图和技术视图中进行了说明，因此本节不再单独展开。

7. 技术视图

本章介绍本 GitHub 仓库 AI 问答系统的全部技术栈选型，覆盖编程语言、开发框架、数据库、中间件、部署工具等内容。

全系统技术栈选型汇总

分类	选型	作用说明
前端开发	React18 + Ant Design5 + Axios	搭建用户前台、管理员后台网页，处理 HTTP 交互

<项目名称>	Version: <1.1>
软件架构文档	Date: <21/07/2026>

分类	选型	作用说明
后端开发	Javal7 + SpringBoot2.7 + MyBatis-Plus	后端业务接口、数据库 ORM 持久化
关系数据库	MySQL 8.0	存储用户、仓库、工单、FAQ 结构化数据
向量数据库	Milvus	代码片段向量存储，实现代码语义检索
AI 服务	通义千问 API	代码问答、Bug 修复、Issue 自动回复、摘要生成
第三方平台	GitHub REST API + Webhook	仓库授权、代码拉取、Issue 消息推送
定时中间件	XXL-Job	定时同步仓库、定时更新向量知识库
通用工具	Hutool、SLF4J	加密、参数校验、日志打印通用方法
部署环境	Docker + Linux 云服务器	容器化打包，统一本地、线上运行环境
开发建模工具	PowerDesigner、IDEA、VSCode、Git	架构绘图、前后端开发、版本管理

8. 数据视图（可选）

8.1 数据视图概述

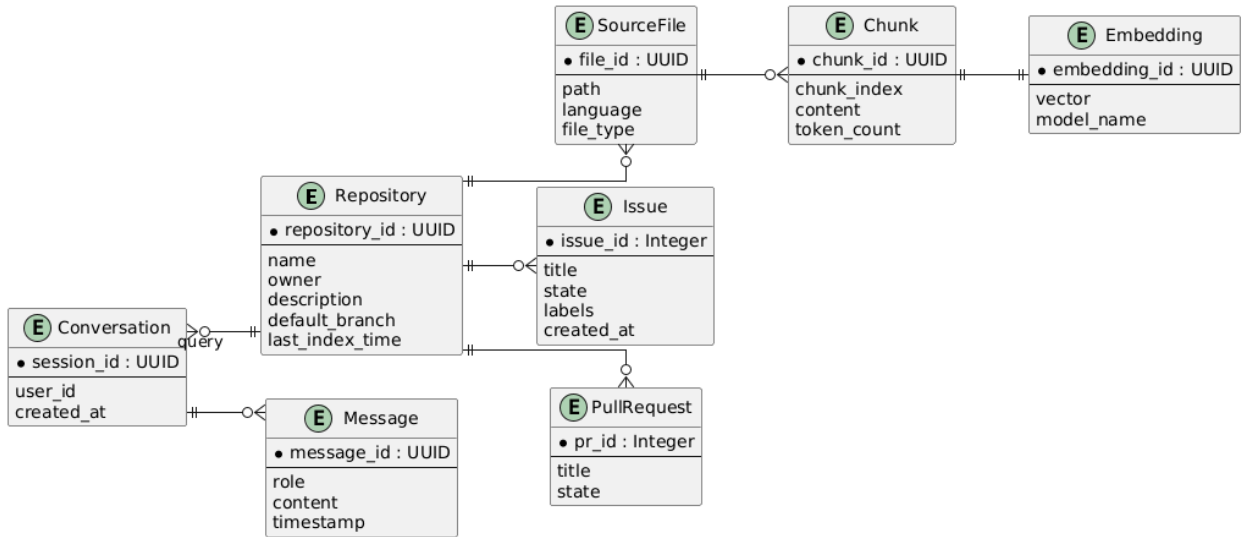
本系统采用检索增强生成（Retrieval-Augmented Generation，RAG）架构，因此需要对 GitHub 仓库中的各类数据进行采集、解析、存储和管理。系统中的永久性数据主要包括 GitHub 仓库信息、文档内容、源码文件、Issue、Pull Request、向量索引以及用户会话信息等。

数据流主要包括以下几个阶段：

- 1. **数据采集：**从 GitHub 获取仓库信息、README、源码、Issue、Pull Request 等数据。
- 2. **数据预处理：**对文本进行清洗、解析和分块（Chunking）。
- 3. **向量化处理：**使用 Embedding 模型将文本转换为向量表示。
- 4. **数据存储：**将原始数据和向量数据分别存储到数据库和向量数据库中。
- 5. **数据检索：**根据用户问题进行相似度检索，获取相关文档片段。
- 6. **结果生成：**将检索结果作为上下文输入大语言模型生成最终回答。

8.2 数据存储结构

<项目名称>	Version: <1.1>
软件架构文档	Date: <21/07/2026>



系统主要维护以下几类数据。

数据类型	主要内容	存储方式	用途
Repository Metadata	仓库名称、描述、作者、分支等	关系数据库	仓库管理
README / Docs	README、Markdown 文档	文档存储	文档检索
Source Code	源代码文件	文档存储	代码问答
Issue 数据	Issue 标题、正文、评论、标签	数据库	Issue 分析
Pull Request 数据	PR 信息及评论	数据库	项目分析
Chunk 数据	文本切分后的片段	数据库	Embedding 输入
Embedding 向量	文本向量	向量数据库	相似度检索
Conversation History	用户历史对话	数据库	多轮对话

8.3 数据组织方式

系统采用分层的数据组织方式。

(1) 原始数据

保存从 GitHub 获取的原始内容，包括：

- Repository 信息
- README
- Markdown 文档
- 源代码
- Issues
- Pull Requests

该层数据保持与 GitHub 仓库一致，便于重新建立索引。

(2) 处理数据

系统对原始数据进行解析后得到：

- 文档切片
- 文件路径
- 所属模块
- 语言类型
- Issue 分类信息

这些数据用于后续 Embedding 建立。

<项目名称>	Version: <1.1>
软件架构文档	Date: <21/07/2026>

（3）向量索引

经过 Embedding 模型处理后生成高维向量。

每个向量包含：

- Chunk ID
- Repository ID
- Embedding Vector
- Metadata（文件名、路径、类型、来源）

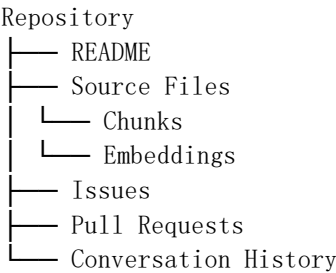
系统利用向量数据库完成 Top-K 相似度检索。

8.4 数据之间的关系

各类数据之间存在如下关联关系：

- 一个 Repository 包含多个 Source File。
- 一个 Repository 包含多个 README 或文档。
- 一个 Repository 包含多个 Issue。
- 一个 Repository 包含多个 Pull Request。
- 一个 Source File 可被划分为多个 Chunk。
- 一个 Chunk 对应一个 Embedding 向量。
- 一个用户会话可关联多个检索结果和多个回答。

其关系可表示如下：



8.5 数据生命周期

系统中的数据生命周期如下：

1. 用户输入 GitHub Repository URL。
2. 系统下载仓库数据。
3. 文档解析器提取文本内容。
4. 文本按照固定长度进行 Chunk 划分。
5. Embedding 模型生成向量。
6. 向量存入向量数据库。
7. 用户提出问题。
8. 检索模块查询相关向量。
9. 检索结果送入 LLM。
10. 生成回答并保存会话记录。

8.6 数据一致性

为了保证数据一致性，系统采用以下策略：

- 当 GitHub 仓库更新时，重新建立对应 Repository 的索引。
- 每个 Repository 使用唯一 Repository ID 进行标识。
- 每个 Chunk 均记录所属文件路径及版本信息。
- Embedding 与 Chunk 保持一一对应关系。

<项目名称>	Version: <1.1>
软件架构文档	Date: <21/07/2026>

- 删除 Repository 时，同时删除对应的 Chunk、Embedding 和缓存数据，避免产生无效索引。

8.7 数据安全

系统对永久性数据采取以下保护措施：

- GitHub Token 等敏感信息采用环境变量管理，不存储于数据库。
- 用户会话数据仅用于当前用户，不向其他用户共享。
- 向量数据库仅保存文本向量及必要的元数据，不保存用户隐私信息。
- 对数据库进行定期备份，并记录系统日志，保证数据的完整性和可恢复性。

9. 算法视图（可选）

系统的核心算法包括 GitHub 仓库数据解析算法、文本切分（Chunking）算法、Embedding 向量生成算法、向量相似度检索算法（Top-K Retrieval）以及检索增强生成（Retrieval-Augmented Generation, RAG）算法。系统首先根据用户问题进行向量检索，获取最相关的文档片段，再将检索结果与用户问题共同构造 Prompt，由大语言模型生成最终回答，从而提高回答的准确性并降低模型幻觉。

9.1 GitHub 仓库数据解析算法

系统通过 GitHub API 获取仓库源码、README、Issue、Pull Request 等数据，并根据不同文件类型采用相应的解析方式。

对于 Markdown 文档，提取标题、正文及代码块；对于源代码文件，按照类、函数、注释等代码结构进行解析；对于 Issue 和 Pull Request，则提取标题、正文、评论及标签等信息。解析后的内容统一转换为文本格式，为后续知识库构建提供输入数据。

9.2 文本切分（Chunking）算法

由于大语言模型和向量模型存在输入长度限制，系统采用文本切分算法将长文档划分为多个较小的文本块（Chunk）。

切分过程中按照自然语言边界（如标题、段落、代码块等）进行划分，并采用一定长度的重叠（Overlap）策略，使相邻文本块保留部分公共内容，避免语义信息在切分过程中丢失，提高后续检索的完整性和准确性。

9.3 Embedding 向量生成算法

系统利用 Embedding 模型将文本转换为固定维度的语义向量。

Embedding 模型能够学习文本的语义特征，使语义相近的文本在向量空间中的距离更近，而语义无关的文本距离较远。生成的向量被存储到 Milvus 等向量数据库中，为后续相似度检索提供基础。

9.4 向量相似度检索算法（Top-K Retrieval）

用户提问后，系统首先将问题转换为向量表示，并与知识库中的文本向量进行相似度计算。

系统采用 Top-K 检索策略，根据向量相似度从知识库中返回最相关的 K 个文本片段作为候选知识。相似度通常采用**余弦相似度（Cosine Similarity）或内积（Inner Product）**进行计算，从而保证检索结果具有较高的语义相关性。

9.5 检索增强生成（RAG）算法

RAG（Retrieval-Augmented Generation）是系统智能问答的核心算法。

传统大语言模型仅依赖模型参数进行回答，容易出现知识过时或模型幻觉等问题。RAG 算法首先利用向量检索获得与用户问题最相关的知识片段，然后将这些检索结果与用户问题共同构造成 Prompt 输入大语言模型，由模型基于外部知识生成最终回答。

通过引入检索结果作为上下文，模型能够结合仓库中的实际代码、文档和 Issue 信息进行回答，提高回答的准确性、可信度和时效性，同时有效降低模型幻觉现象。

<项目名称>	Version: <1.1>
软件架构文档	Date: <21/07/2026>

10. 性能视图（可选）

本项目为课程级小型单体应用，无高并发、海量数据、高吞吐的硬性线上性能指标要求，未针对性能做专项架构设计（如分布式缓存、分库分表、负载均衡、流量削峰等）。基础性能保障已通过异步任务流水线、向量数据库选型、分层解耦架构实现，相关内容已在技术视图、进程视图中覆盖，无需独立成章。

11. 可靠性视图（可选）

本系统为课程演示项目，无 7×24 小时高可用、多副本容灾、故障自动转移等硬性可靠性要求；仅依赖框架自带的异常捕获、日志记录能力做基础故障处理，未设计服务降级、熔断重试、集群容错等专项可靠性架构，相关基础保障能力已在公共工具模块说明，无需独立章节展开。

12. 安全性视图（可选）

系统核心安全设计（用户登录鉴权、GitHub 授权凭证加密存储、接口参数校验、操作日志留存）已在业务服务层、公共工具模块中完整说明。项目无等保合规、多租户数据隔离、细粒度权限管控、防渗透攻击等复杂安全架构需求，未做专项安全架构设计，无需单独成章。

13. 易用性视图（可选）

前端页面的角色拆分、功能分层、交互流程设计，已在技术视图的前端构件小节详细描述，页面功能贴合普通用户与管理员的使用习惯。项目无针对易用性的架构级专项设计（如无障碍适配、多语言架构、个性化界面配置体系等），交互相关内容已在前端模块说明中覆盖，无需独立视图。

14. 可维护性视图（可选）

14.1 设计目标

系统采用模块化、分层（Tier）架构设计，以降低模块之间的耦合度，提高系统的可维护性和可扩展性。当业务需求发生变化或第三方服务升级时，能够将修改范围控制在对应模块内，减少对其他模块的影响，便于后续功能迭代和长期维护。

14.2 可维护性设计

1. 分层单向依赖

系统采用 Presentation Tier、Business Tier、Data Tier 三层架构，各层仅依赖下一层提供的接口。其中 Presentation Tier 负责页面展示及用户交互；Business Tier 负责仓库管理、知识库构建、RAG 问答、Issue 分析等业务逻辑；Data Tier 负责 MySQL、Redis、Milvus 以及 GitHub 数据访问。这种单向依赖方式避免了循环依赖，使各层职责明确。当修改数据库实现或业务逻辑时，不会影响表示层，从而降低维护成本。

系统涉及多个第三方平台，包括 GitHub API、大语言模型 API 以及向量数据库接口。为了减少业务代码与外部平台之间的耦合，系统将所有第三方接口统一封装为独立的 Service 或 Adapter，由业务层通过统一接口进行调用。

2. 第三方服务 API 统一封装

系统涉及多个第三方平台，包括 GitHub API、大语言模型 API 以及向量数据库接口。为了减少业务代码与外部平台之间的耦合，系统将所有第三方接口统一封装为独立的 Service 或 Adapter，由业务层通过统一接口进行调用。

统一封装后，当 GitHub API 版本升级、更换大语言模型供应商或替换向量数据库时，仅需修改对应接口实现，而无需调整业务逻辑，提高了系统的可维护性。

3. 公共模块抽离

<项目名称>	Version: <1.1>
软件架构文档	Date: <21/07/2026>

系统将多个通用功能抽取为公共模块，包括：配置管理，日志管理，全局异常处理，工具类，通用数据对象。公共模块供各业务模块统一调用，避免重复实现，提高代码复用率，也降低了维护成本。

4. 业务模块解耦

系统按照业务职责划分多个独立模块，包括：用户管理模块，仓库管理模块，知识库管理模块，AI 问答模块，Issue 分析模块，后台管理模块
各模块通过统一 Service 接口进行交互，不直接依赖彼此内部实现。当新增功能或修改业务逻辑时，只需维护对应模块即可，不影响其他模块运行。

14.3 可维护性分析

系统通过分层架构、第三方 API 统一封装、公共模块抽离以及业务模块解耦等设计，提高了代码复用率和模块独立性。各模块职责清晰，修改影响范围较小，能够较好地支持后续功能扩展、第三方服务升级以及系统长期演进，满足项目的可维护性要求。

